

Database Systems for Application Engineers

Controlled RAG Evaluation Corpus - Book 2

Purpose

This synthetic mini-book is designed for a beginner-friendly RAG engineering laboratory. It contains stable facts, overlapping terminology, and controlled distractors so retrieval and generation changes can be evaluated reproducibly.

Corpus rule: keep this file unchanged once the baseline evaluation dataset has been created.

Chapter 1: 1. Relational Foundations

A relational database organizes information into tables made of rows and columns. A primary key uniquely identifies a row. A foreign key references a key in another table and expresses a relationship between records.

Normalization reduces unnecessary duplication by separating facts into related tables. It can improve consistency, although highly normalized schemas sometimes require more joins when reading data.

SQL is a declarative language: a query describes the desired result rather than prescribing every physical execution step. The database optimizer chooses an execution plan.

Chapter 2: 2. Transactions and ACID

A transaction groups operations into a logical unit of work. Atomicity means the transaction's changes succeed as a unit or are rolled back. Consistency means valid constraints should remain satisfied. Isolation controls interactions between concurrent transactions. Durability means committed changes survive expected failures.

A bank transfer illustrates atomicity. Debiting one account without crediting the other would leave the system inconsistent, so both changes should belong to one transaction.

Isolation levels trade stronger concurrency guarantees for performance. Applications should choose an isolation level based on anomalies they must prevent rather than always assuming the strongest level is necessary.

Chapter 3: 3. Indexes and Query Performance

An index is an auxiliary data structure that can accelerate lookup at the cost of additional storage and write maintenance. Indexing every column is usually wasteful.

A composite index contains multiple columns, and column order can affect which queries use it efficiently. Query plans should be inspected rather than assuming an index will always be selected.

Database caching and vector similarity search are separate concepts. A conventional SQL index typically accelerates exact, range, or ordered lookup. A vector index is designed for nearest-neighbor search over embeddings.

Chapter 4: 4. Vector Databases and Metadata

A vector database stores embeddings and supports nearest-neighbor retrieval. Each vector is commonly associated with the original text or object and metadata such as source, document name, page, category, tenant, or timestamp.

Metadata filtering narrows the candidate set using structured attributes. In a library RAG system, a filter can restrict retrieval to a particular book before semantic similarity is applied.

ChromaDB is commonly used as a vector store in local RAG prototypes. Persistence allows indexed data to remain available across application restarts instead of recomputing every embedding at startup.

Chapter 5: 5. Security and Multi-Tenancy

Database credentials should be stored outside source code and granted only the permissions the application needs. Least privilege reduces damage if a credential is exposed.

In a multi-tenant retrieval system, metadata filtering is not merely a relevance feature; it can be part of authorization enforcement. A user must not retrieve another tenant's private chunks simply because their embeddings are semantically similar.

Backups protect availability but are not a substitute for access control, encryption, auditing, or application-level authorization.

Evaluation Notes

This book is synthetic and intentionally compact. Facts are written explicitly so they can support reference answers, retrieval-context checks, groundedness tests, citation experiments, and regression tests. Some passages are deliberate distractors and are labeled as such.